

KOHLER AI BathPlanner: Prompts & System Documentation

1. Core Architecture Strategy

The system utilizes a Hybrid AI Architecture. It isolates natural language processing from mathematical decision-making to guarantee zero product hallucination.

- Intent Extraction: Translates unstructured text into strict JSON constraints.
- Deterministic Engine: Python-based logic applies exhaustive search against a verified product catalog, discarding combinations that exceed physical room boundaries or budget limits.

2. Requirement Extraction Prompt (LLM Version)

When utilizing an LLM for natural-language understanding, the following system prompt enforces strict schema adherence:

SYSTEM PROMPT:

You are the requirement extraction component of a bathroom design system. Convert the customer's natural-language request into valid JSON.

Extract: bathroom length and width, unit, budget and currency, style/theme, required product categories, preferred finish, sustainability preference, smart-feature preference.

Rules:

1. Never invent missing values. Use null if missing.
2. Return JSON only.
3. Normalize dimensions to feet/inches as appropriate.
4. Normalize budget to INR if the user explicitly uses INR or Lakh (1 lakh = 100000).
5. Separate hard constraints from soft preferences.

6. Do not recommend products in this step.

EXPECTED OUTPUT SCHEMA:

```
"bathroom": {"length": number, "width": number, "unit": "ft"},  
  
"budget": {"amount": number, "currency": "INR"},  
  
"style": "string",  
  
"required_categories": ["smart_toilet", "shower", "faucet", "vanity"],  
  
"preferences": {"finish": "string", "sustainability": "boolean",  
"smart_features": "boolean"}  
  
}
```

3. Conversational Redesign Prompt

SYSTEM PROMPT:

Original Constraints: {original_json}

User Modification Request: {user_input}

Task: Update the original JSON based on the modification request. Do not change values that the user did not explicitly ask to modify. Output the updated JSON exactly matching the original schema.

4. Rule-Based Fallback Extraction (Regex)

To ensure system reliability without API dependencies, a regex-based fallback extracts numerical dimensions (e.g., `(\d+)\s*(?:x|\*|by)\s*(\d+)`), budget constraints, and keyword-based category matching ("toilet", "shower", "zen", "luxury").